



CCODEL
RAISING A NEW GENERATION OF
GLOBAL LEADERS

Computer Programming I (Java 1)

COS221

Mr. Otavie Okuoyo

Phd (in view Computer Science)

College of Science and Technology



Visit Our Website

ccodel.covenantuniversity.edu.ng



MODULE TEN: EXCEPTION HANDLING AND FILE I/O

UNIT ONE: EXCEPTION HANDLING AND FILE I/O

Learning Objectives

By the end of this unit, you will be able to:

- **define exceptions and differentiate them from errors.**
- **explain the need for exception handling in software development.**
- **demonstrate the syntax and use of try, catch, and finally constructs.**
- **handle multiple exceptions using appropriate programming techniques.**
- **perform file operations including reading, and writing using appropriate I/O classes.**

Learning Outcomes

At the end of this unit, you should be able to:

- **understand the concept and importance of exception handling in programming.**
- **identify different types of exceptions and errors in code.**
- **implement exception handling mechanisms using try, catch, and finally blocks.**
- **write programs that can gracefully handle runtime errors without abrupt termination.**
- **perform file operations (read, write) using I/O classes.**

Introduction

- In any programming environment, unexpected events or errors can occur during the execution of a program, leading to abrupt termination if not managed properly. These events, known as exceptions, can arise from various causes such as invalid user input, file access issues, or logical errors.
- Exception handling is a fundamental programming technique designed to anticipate, detect, and respond to such errors gracefully. This unit introduces the concepts and practical methods of exception handling, enabling developers to write fault-tolerant and reliable software.

Exception and Why Handle it?

- An exception is an event that disrupts the normal flow of a programme. It is an object representing an error condition.
- The following are the reasons for handling exception:
 - i. Exception prevents program crashes.
 - ii. It provides a user-friendly error message.
 - iii. Handle unforeseen situations at runtime.

Types of Exception

The following are types of Exceptions:

- 1. Checked Exceptions: Checked at compile-time. They must be either caught or declared to be thrown. Example: IOException, FileNotFoundException, SQLException, ClassNotFoundException, etc.**
- 2. Unchecked Exceptions: Occur at runtime. They do not need to be declared or caught. Example: ArithmeticException, NullPointerException, ArrayIndexOutOfBoundsException, etc.**

Exception

vs

Error

- 1. An event that disrupts normal flow of a program, which can be caught and handled.**
- 2. Subclass of `java.lang.Exception`**
- 3. Examples include `IOException`, `SQLException`, `ArithmeticException`, `NullPointerException`**

- 1. Serious problems that occur beyond the control of the application and usually cannot be handled.**
- 2. Subclass of `java.lang.Error`**
- 3. Examples include `OutOfMemoryError`, `StackOverflowError`, `VirtualMachineError`**

Exception

vs

Error (2)

- 4. Checked exceptions must be declared or caught; unchecked are runtime exceptions.**
- 5. Can be caught and handled using try-catch blocks**
- 6. Possible to recover and continue program execution**

- 4. Always unchecked**
- 5. Generally not caught; indicates serious JVM issues**
- 6. Usually not recoverable; leads to program termination**

Common Exceptions and Errors

Type	Example	Description
Checked Exception	IOException	Input/Output error, e.g. file not found.
Checked Exception	ClassNotFoundException	Class not found during runtime
Unchecked Exception	NullPointerException	Accessing an object through a null reference
Unchecked Exception	ArithmeticException	Division by zero
Unchecked Exception	ArrayIndexOutOfBoundsException	Accessing invalid array index
Error	OutOfMemoryError	JVM runs out of memory
Error	StackOverflowError	Stack memory is exhausted due to deep recursion

Handling Exception - *try-catch Block*

The syntax for try-catch block is:

```
try {  
    // Risky mode  
} catch (ExceptionType e) {  
    // Handle Exception  
}
```

```
public class TryCatch {  
    public static void main(String[] args) {  
        try {  
            int result = 10 / 0;  
            System.out.println("Result: " + result);  
        } catch (ArithmeticException e) {  
            System.out.println("Cannot divide by zero!");  
        }  
    }  
}
```

The code will
output:

***Cannot divide by
zero!***

Handling Exception - *finally* Block

The *finally* block of code always executes whether an exception occurs or not. It is used for cleanup (e.g. closing files).

```
try {  
    int result = 10 / 0; // Causes ArithmeticException  
} catch (ArithmeticException e) {  
    System.out.println("Error: Division by zero!");  
} finally {  
    System.out.println("This block always runs.");  
}
```

The output of the code is:

Error: Division by zero!
This block always runs.

Handling Exception - *Multiple catch Block*

- Multiple catch blocks allow handling different exception types separately.
- The try block contains code that may throw exceptions, and each catch block handles a specific exception.
- Only the matching catch block runs, enabling precise and clear error management.

Handling Exception - *Multiple catch Block (2)*

```
public class ExceptionHandlingDemo {  
    public static void main(String[] args) {  
        try {  
            int[] arr = new int[5];  
            arr[10] = 50; // Throws ArrayIndexOutOfBoundsException  
        } catch (ArithmeticException e) {  
            System.out.println("Math error");  
        } catch (ArrayIndexOutOfBoundsException e) {  
            System.out.println("Array index out of bounds!");  
        }  
    }  
}
```

The output of the code is *Array index out of bounds!*

Handling Exception - *Using throw*

- The **throw** keyword is used to explicitly throw an exception.
- When used, it terminates the current method execution and passes control to the nearest matching catch block.

```
public static void registerUser(int age) {  
    if (age < 18) {  
        throw new IllegalArgumentException("You must be 18.");  
    } else {  
        System.out.println("Registration successful.");  
    }  
}
```

Handling Exception - *throw* Ex - 1

```
public class UserRegistration {  
    public static void registerUser(int age) {  
        if (age < 18) {  
            throw new IllegalArgumentException("You must be 18.");  
        } else {  
            System.out.println("Registration successful.");  
        }  
    }  
  
    public static void main(String[] args) {  
        registerUser(16);  
    }  
}
```

Handling Exception - *throw Ex - 1 (2)*

Given the code above, the static method, registerUser(), takes an integer, age. It throws an IllegalArgumentException if age is less than 18, stopping the execution with an error. The above code will output:

Exception in thread "main"

java.lang.IllegalArgumentException: You must be 18.

at

UserRegistration.registerUser(UserRegistration.java:4)

at UserRegistration.main(UserRegistration.java:11)

Handling Exception - *throw* Ex - 2

```
public class DivisionMain {  
    public static void main(String[] args) {  
        try {  
            divide(10, 0);  
        } catch (ArithmeticException e) {  
            System.out.println("Caught exception: " + e.getMessage());  
        }  
    }  
  
    static void divide(int a, int b) {  
        if (b == 0) {  
            throw new ArithmeticException("Cannot divide by zero");  
        }  
        System.out.println("Result: " + a / b);  
    }  
}
```

Handling Exception - *throw Ex - 2 (2)*

- The method throws an `ArithmeticException` if the divisor is zero, preventing division by zero errors.
- In the main method, `divide(10, 0)` is called inside a try block, and when the exception is thrown, it is caught by the catch block, which prints an error message instead of crashing the program.
- The output of the code is:

Caught exception: Cannot divide by zero

Handling Exception - *Using throws*

- The **throws** keyword is used in a method declaration to indicate that the method might throw one or more exceptions.
- It tells the caller of the method that they need to handle or further declare these exceptions.
- Unlike **throw**, which actually throws an exception, **throws** is only a part of the method signature signaling possible exceptions.

Handling Exception - *Using throws* (2)

```
public class ThrowsDivision {  
    public static int divide(int a, int b) throws ArithmeticException {  
        return a / b;  
    }  
  
    public static void main(String[] args) {  
        try {  
            int result = divide(10, 0);  
            System.out.println("Result: " + result);  
        } catch (ArithmeticException e) {  
            System.out.println("Error: Cannot divide by zero.");  
        }  
    }  
}
```

The output of the code is as follows:

Error: Cannot divide by zero.

Custom Exception

- A custom exception is a user-defined exception class that extends either `Exception` (for checked exceptions) or `RuntimeException` (for unchecked exceptions).
- It allows you to create specific error types that make your code more readable and expressive.
- Custom exceptions help communicate precise problems that standard exceptions can't describe well.

Custom Exception - Example 1

```
class InvalidAgeException extends Exception {  
    public InvalidAgeException(String message) {  
        super(message);  
    }  
}  
  
public class CustomThrow {  
    public static void main(String[] args) throws InvalidAgeException {  
        int age = 15;  
  
        if (age < 18) {  
            throw new InvalidAgeException("Age must be 18");  
        }  
    }  
}
```

The code will output the following:

Exception in thread "main" InvalidAgeException: Age must be 18 at CustomThrow.main(CustomThrow.java:12)

Custom Exception - Example 2

```
class InvalidScoreException extends RuntimeException {  
    public InvalidScoreException(String message) {  
        super(message);  
    }  
}  
  
public class CustomException {  
    public static void checkScore(int score) {  
        if (score < 0 || score > 100) {  
            throw new InvalidScoreException("Score must be between 0 and 100.");  
        }  
        System.out.println("Valid score: " + score);  
    }  
  
    public static void main(String[] args) {  
        try {  
            checkScore(85);    // Valid score  
            checkScore(150);   // Invalid score, will throw exception  
        } catch (InvalidScoreException e) {  
            System.out.println("Caught exception: " + e.getMessage());  
        }  
    }  
}
```

Custom Exception - Example 2 (2)

- The code above defines a custom unchecked exception `InvalidScoreException` that is thrown when a score is outside the valid range (0 to 100).
- The `checkScore` method checks the score and throws the exception if invalid. In main, the method is tested with both valid and invalid scores, and any thrown exceptions are caught and handled gracefully.
- The code will output the following:

Valid score: 85

Caught exception: Score must be between 0 and 100.

File Input and Output (I/O)

File I/O refers to reading data from, writing data to files and carrying out other file operations using classes from the java.io or java.nio packages. The following are ways to write to and read from a file:

Write to a file:

- ***FileWriter***
- ***BufferedWriter***
- ***PrintWriter***
- ***Files.write()***

Read from a file:

- ***FileReader***
- ***BufferedReader***
- ***Scanner***
- ***Files.readAllLines()/Files.readString()***

Writing with FileWriter - Ex 1

```
import java.io.FileWriter;
import java.io.IOException;

public class WriteToFileExample {
    public static void main(String[] args) {
        try {
            FileWriter wr = new FileWriter("test.txt");
            wr.write("Hello Boss! This s a test.\n");
            wr.write("Writing to a file in Java is easy");
            wr.close();
        } catch (IOException e) {
            System.out.println("An error occurred: " + e.getMessage());
        }
    }
}
```

The code writes text to a file named ***test.txt*** using ***FileWriter***. The ***write()*** method is used to writes two lines of text to the file.

Writing with PrintWriter - Ex 2

```
import java.io.PrintWriter;  
import java.io.IOException;  
  
public class PrintWriterOne {  
    public static void main(String[] args) {  
        try (PrintWriter pw = new PrintWriter("output.txt")) {  
            pw.print("This is ");  
            pw.println("on the same line");  
            pw.printf("The number again is %d", 221);  
        } catch (IOException e) {  
            System.out.println("Error Writing: " + e.getMessage());  
        }  
    }  
}
```

The code uses *PrintWriter* to write to *output.txt* with *print()*, *println()*, and *printf()*. The content of *output.txt* is:

This is on the same line
The number again is 221

Writing with BufferedWriter - Ex 3

```
import java.io.*;
public class BufferedWriterOne {
    public static void main(String[] args) {
        try(BufferedWriter bw = new BufferedWriter(
            new FileWriter("buffer.txt"))) {
            bw.write("Buffer writing is efficient");
            bw.write("\nSecond writing with buffer");
            System.out.println("Writing was successful!");
        } catch (IOException e) {
            System.err.println("Error: " + e.getMessage());
        }
    }
}
```

This program writes to the file "**buffer.txt**" using **BufferedWriter**, which improves efficiency by buffering characters.

Writing with NIO Files.write() - Ex 4

```
import java.nio.file.*;
import java.io.IOException;

public class NIOFileWriter {
    public static void main(String[] args) throws IOException {
        Path path = Paths.get("nio.txt");
        Files.write(path, "Hello World".getBytes());
        System.out.println("File successfully written!");
    }
}
```

The program writes "*Hello World*" to a file named *nio.txt* using the *NIO Files.write()* method. It creates the file if it does not exist.

Reading Using FileReader - Ex 1

```
import java.io.FileReader;
import java.io.IOException;
public class FileReaderExample {
    public static void main(String[] args) {
        try (FileReader fr = new FileReader("output.txt")) {
            int ch;
            while ((ch = fr.read()) != -1) {
                System.out.print((char) ch);
            }
        } catch (IOException e) {
            System.err.println("Error: " + e.getMessage());
        }
    }
}
```

The code reads from “**output.txt**” using **FileReader**. Each character is cast from its integer value and printed until **read()** returns **-1** (end of file).

Reading Using BufferedReader - Ex 2

```
import java.io.*;
public class BufferedReaderExample {
    public static void main(String[] args) {
        try (BufferedReader br = new BufferedReader(
            new FileReader("output.txt"))) {
            String line;
            while ((line = br.readLine()) != null) {
                System.out.println(line);
            }
        } catch (IOException e) {
            System.err.println("ReadError: " + e.getMessage());
        }
    }
}
```

This code reads “**output.txt**” line by line using **BufferedReader**. It prints each line until **readLine()** returns **null**, indicating the end of the file.

Reading Using Scanner - Ex 3

```
import java.io.FileNotFoundException;
import java.io.File;
import java.util.Scanner;

public class ScannerReader {
    public static void main(String[] args) {
        try (Scanner sc = new Scanner(
            new File("output.txt"))) {
            while (sc.hasNextLine()) {
                System.out.println(sc.nextLine());
            }
        } catch (FileNotFoundException e) {
            e.printStackTrace();
        }
    }
}
```

This code uses a ***Scanner*** to read “***output.txt***” line by line. It prints each line until there are no more lines to read.

The Exception, ***FileNotFoundException***, is handled if the file does not exist.

Reading with Files.readAllLines() - Ex 4

```
import java.nio.file.Files;
import java.nio.file.Paths;
import java.io.IOException;
import java.util.List;

public class NIOReaderExample {
    public static void main(String[] args) {
        try {
            List<String> lines = Files.readAllLines(
                Paths.get("example.txt"));
            for (String line : lines) {
                System.out.println(line);
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

This code reads all lines from “*example.txt*” at once using *Files.readAllLines()* and prints each line.

It handles *IOException* if reading fails.

In-Text Questions (ITQs)

- 1. What are the main keywords used in Java for handling exceptions, and how do they differ?**
- 2. What is the purpose of the finally block?**
- 3. What are custom exceptions, and how are they implemented in Java?**
- 4. Why should exception handling be used with file operations?**
- 5. Why is BufferedWriter preferred over FileWriter for writing large text files?**

In-Text Answers (ITAs)

1. The main keywords are:
 - a. **try**: Defines a block of code that might throw exceptions.
 - b. **catch**: Catches and handles specific exceptions thrown in the try block.
 - c. **finally**: Contains code that always executes after try-catch blocks for cleanup purposes.
 - d. **throw**: Used to manually throw an exception.
 - e. **throws**: Declares the exceptions that a method might throw.

In-Text Answers (ITAs) (2)

- 2. To execute cleanup code, such as closing files, regardless of whether an exception occurred.**
- 3. Custom exceptions are user-defined exception classes that extend Exception or RuntimeException. They allow developers to create specific exceptions tailored to the needs of their applications, enhancing clarity and error handling.**
- 4. Because file operations can fail due to issues like missing files or access restrictions, which may throw IOException.**
- 5. Because it buffers the output and reduces the number of write operations, improving performance.**

Conclusion

- **Exception handling is a vital part of modern programming that ensures software robustness and user-friendly error management.**
- **By mastering the mechanisms to catch and manage exceptions, programmers can prevent crashes and maintain control over program flow even when errors occur.**
- **This unit has provided a foundational understanding and practical skills to effectively handle exceptions, which will be crucial as we advance to more complex topics such as file input/output operations.**

Summary

- **Exceptions are runtime anomalies that disrupt normal program flow. Exception handling uses try, catch, and finally blocks to detect and manage errors.**
- **Proper handling improves program stability and user experience.**
- **Multiple exceptions can be managed through structured catch blocks.**
- **Understanding and implementing exception handling is essential for writing reliable code.**